

Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology



Department of Mechatronics Engineering

Course No.: MTE 3104

Course Title: Microcontroller & Interfacing Sessional

Experiment No.: 1

Experiment Name: Study of ATmega32 Architecture and Implementation of LED Blinking Using PORTC

Remarks

Submitted By:

Bayazid Habib Siddiquee

Roll:2208053

Department of Mechatronics, RUET

Submitted To:

Md. Sakib Hassan Chowdhury,

Lecturer,

Department of Mechatronics, RUET

Date of Submission: 13/1/2026

Date of Experiment:

Experiment No.: 1

Experiment Name: Study of ATmega32 Architecture and Implementation of LED Blinking Using PORTC

Objectives:

- To study the basic architecture of the AVR ATmega32 microcontroller.
- To understand the function of I/O ports in the ATmega32.
- To write and execute a C program to control LEDs connected to PORTC.
- To observe the behavior of digital output pins by blinking LEDs in a defined pattern.

Apparatus Required:

- Proteus Simulation Software
- Atmel Studio

Introduction:

The ATmega32 is an 8-bit AVR microcontroller based on a modified Harvard architecture. It has separate memories for program and data, which allows faster instruction execution. ATmega32 provides four 8-bit I/O ports: PORTA, PORTB, PORTC, and PORTD. Each port has: A Data Direction Register (DDRx) to configure input or output. A PORTx register to write output values. A PINx register to read inputs. In this experiment, PORTC was used as an output port to drive eight LEDs. Each bit of PORTC corresponds to one LED [1]. Writing logic 1 turns an LED ON, and writing logic 0 turns it OFF. A delay was introduced using the `_delay_ms()` function to make the blinking pattern visible to the human eye [1].

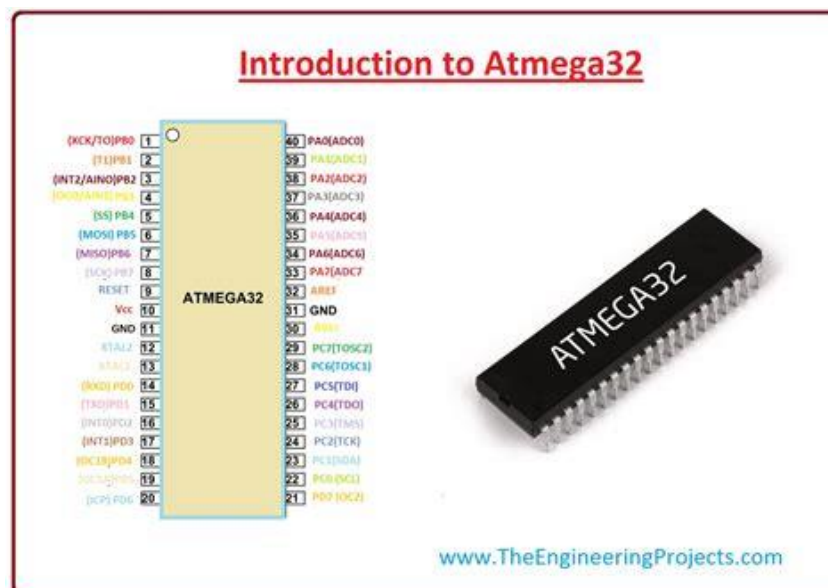


Fig 1: ATmega32 and its pin numbers from perumperindo.co.id

The ATmega32 microcontroller operates at a configurable clock frequency defined by the macro `F_CPU`. This frequency is used by delay functions to calculate accurate timing intervals [1]. The Data Direction Register (DDR) controls whether a port pin acts as an input or output. Setting a DDR bit to logic '1' configures the corresponding pin as an output [2]. PORT registers are used to write digital data to output pins. By assigning different binary values to a port, various LED patterns can be produced. Software delays are generated using the `_delay_ms()` function from the `<util/delay.h>` library, which creates precise delays based on the system clock [3].

Code:

```
/*
 * GccApplication4.c
 * Created: 19/1/2026 10:45:01 am
 * Author : Bayazid_HS
 */
#define F_CPU 8000000UL
#include <util/delay.h>
#include <avr/io.h>
int main(void){
    DDRC = 0xFF;
    while (1){
        PORTC = 0x00;
        int c[] = {0b00000001, 0b00000010, 0b00000100,
0b00000000,
0b00000000, 0b00100000, 0b01000000, 0b10000000};
        for (int x = 0;x<8;x++){
            PORTC = c[x];
            _delay_ms(100);
        }
    }
}
```

Workings:

- DDRC is set to 0xFF, which configures all PORTC pins as outputs.
- First, the value 0x07 is written to PORTC, turning ON PC0, PC1, and PC2.
- After a delay of 0.1 second, all LEDs are turned OFF.
- Next, 0x38 is written to PORTC, turning ON PC3, PC4, and PC5.
- This pattern repeats continuously.

Circuit Diagram:

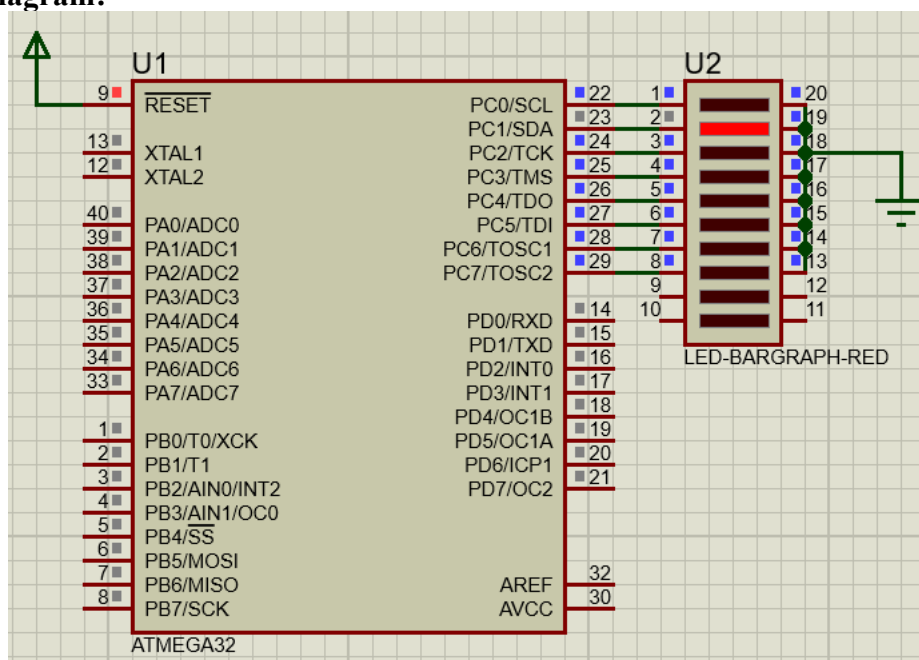


Fig 2: Circuit Diagram of the circuit using bar graph LED and ATmega32 in Proteus

Result:

The experimental results show that the LEDs connected to PORTC light up according to the predefined binary sequence. Each LED pattern is displayed for approximately 100 milliseconds before transitioning to the next. The sequence repeats continuously without error, confirming correct port configuration and timing control.

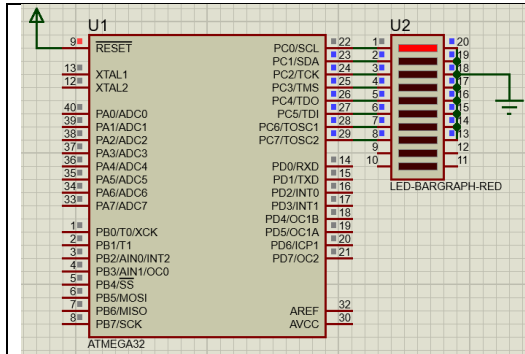


Fig 3: At 1st loop

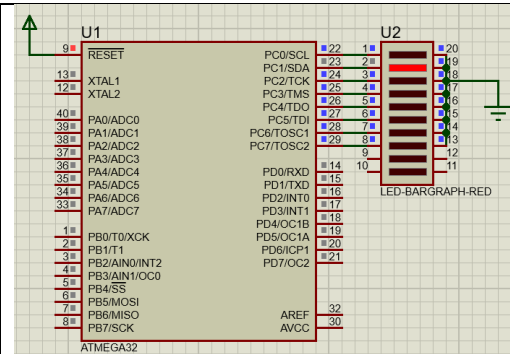


Fig 4: At 2nd loop

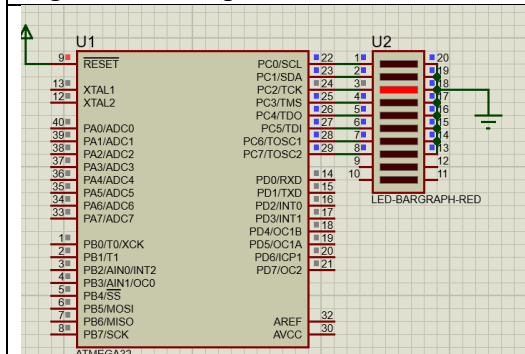


Fig 5: At 3rd loop

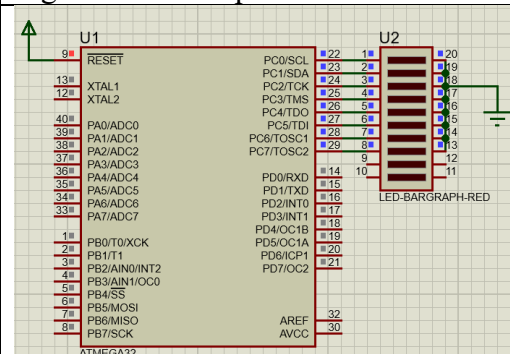


Fig 6: At 4th loop

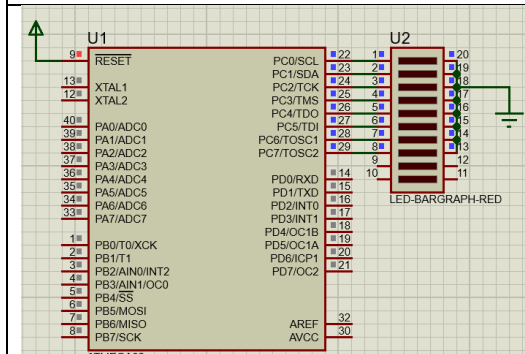


Fig 7: At 5th loop

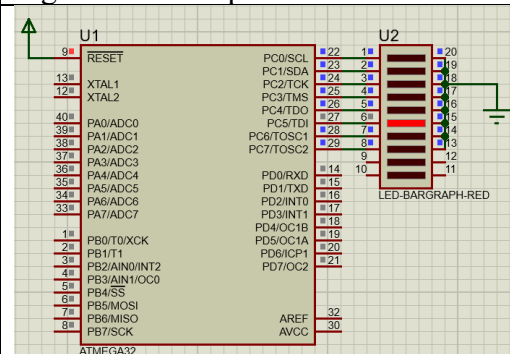


Fig 8: At 6th loop

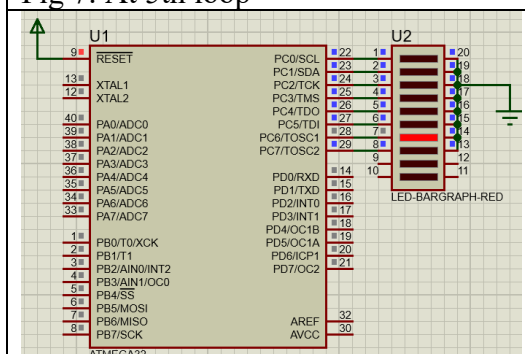


Fig 9: At 7th loop

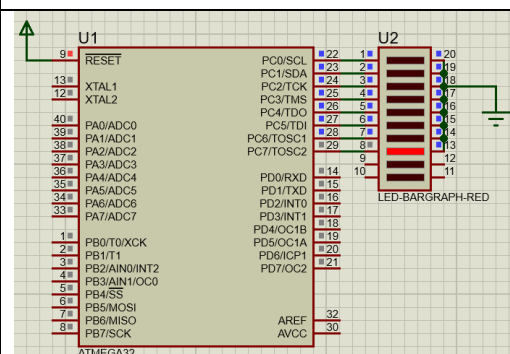


Fig 10: At 8th loop

Discussion:

During this experiment, I faced difficulties in understanding how addresses, registers, and ports actually work at the hardware level. Before reading the AVR book, I was confused about how writing a number to a register could control physical pins, and I did not clearly understand the relationship between memory-mapped I/O and actual hardware pins. While coding in C, I also encountered problems in using bit operations correctly and in selecting the right values to turn on specific LEDs. After studying the introductory chapter of the AVR microcontroller book, I gained a much clearer understanding of how the internal architecture of the ATmega32 works, how the data direction registers configure pins as inputs or outputs, and how the PORT registers directly map to physical pins. This theoretical knowledge helped me correct my mistakes and write more accurate and efficient code.

Conclusion:

This lab work successfully demonstrated the basic working of the ATmega32 microcontroller and its digital I/O system. Reading the introductory chapter of the AVR microcontroller book was essential in helping me understand how registers, memory, and ports operate internally. Although I initially faced problems with addressing and C programming, the combination of theory and practical implementation allowed me to overcome these difficulties. This experiment provided a strong foundation for more advanced embedded system applications such as interfacing sensors, displays, and communication modules.

Reference:

1. Microchip Technology Inc., *ATmega32 Datasheet*, AVR 8-bit Microcontroller.
2. Muhammad Ali Mazidi, *AVR Microcontroller and Embedded Systems*, Pearson Education.
3. AVR Libc Documentation, *Delay Functions*, AVR-GCC User Manual.